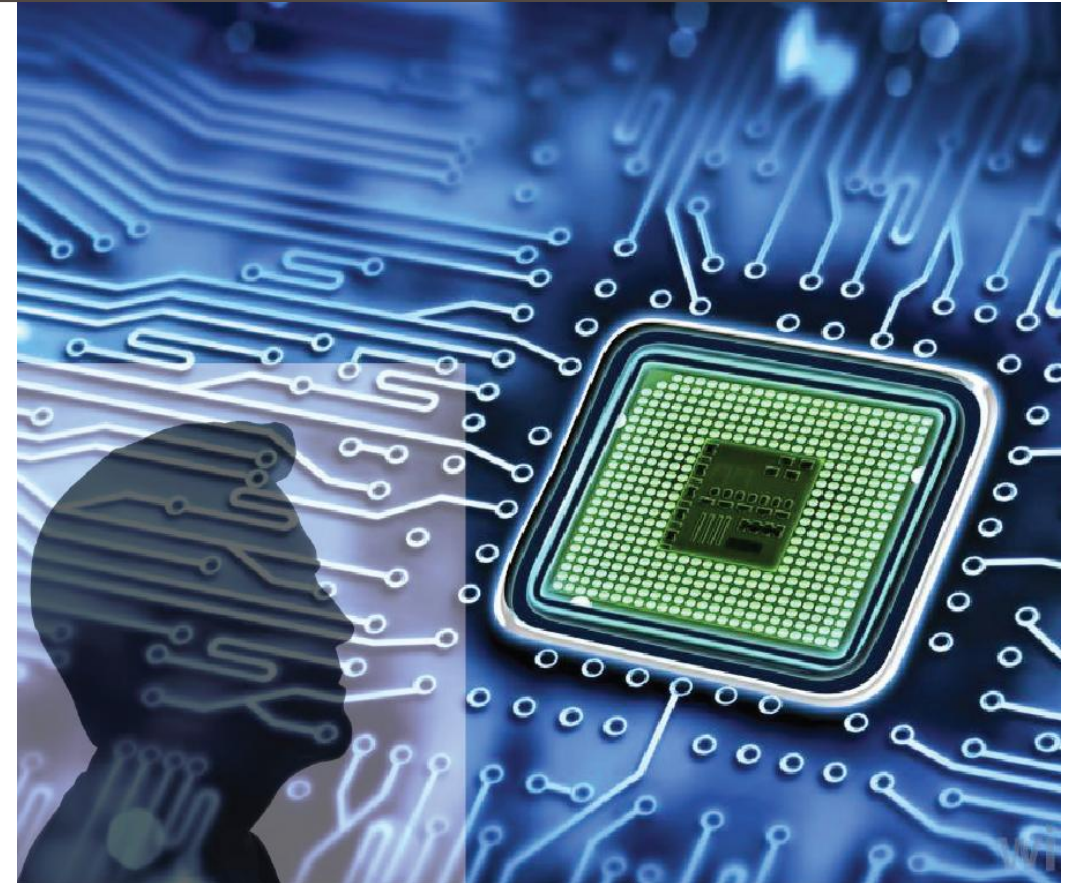


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-5 Lecture-1 Partitioning

Sneh Saurabh
9th September, 2025



Partitioning

Partitioning: Key Aspects

Key aspects:

- Functional Decomposition
- Architecture Description
- Partitioning approaches
- Hardware partition, Software Partition, Communication

Partitioning: Approach

One of the approaches is successive refinement

Successive Refinement:

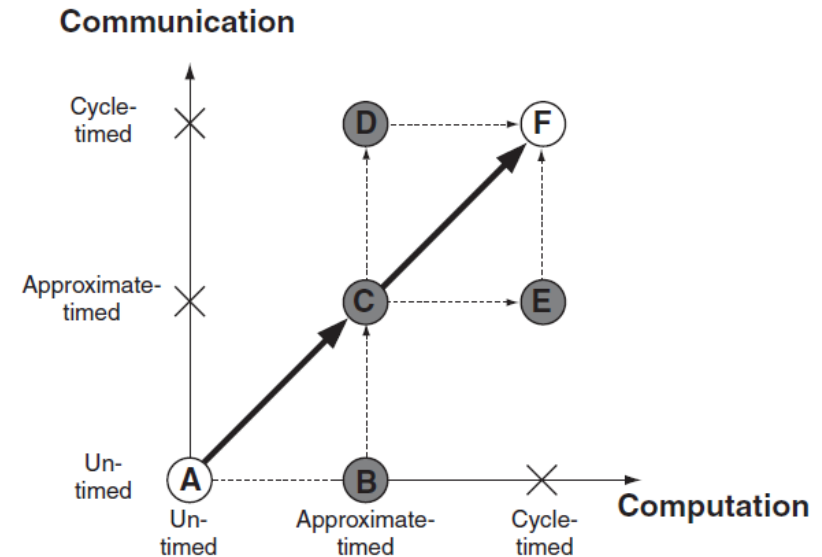
- **Refinement:** introduce additional information or making specification more precise
- Each step refines and transforms the functional model into a mapped multi-abstraction level model

- Two independent aspects — communication and computation
- Refining: time-independent to cycle-accurate

- **Model A:** specification model

- **Model B:** add architectural information and the related functional timing behavior (component assembly model)

- **Model C:** add timing for communication (bus arbitration model)



- **Model D:** cycle-accurate bus-functional model

- **Model E:** cycle-accurate computation model

- **Model F:** final RTL model

Partitioning Implementation

Hardware considerations:

- Refinement should ensure system-level scheduling requirements
- Constraints of control and data dependencies of concurrent processes is critical
- Reconfigurable hardware more being adopted

Software considerations:

- Software partitioning across multiple processors and memories
 - May be further partitioned across tasks, processes, and threads
 - Scheduling algorithm must be chosen
 - Communication burdens that must be considered

Memory considerations:

- Distributing code and data across multiple memory subsystems
- **Caching techniques:** Impact of assigning data structures by their spatial and temporal access patterns

Communication considerations:

- Performance and robustness
- Two approaches:
 - **Template instantiation:** matches one or more templates against functional and performance requirements, selects those that best match the system needs.
 - **Interface synthesis:** FSM models

Platform-based Designs

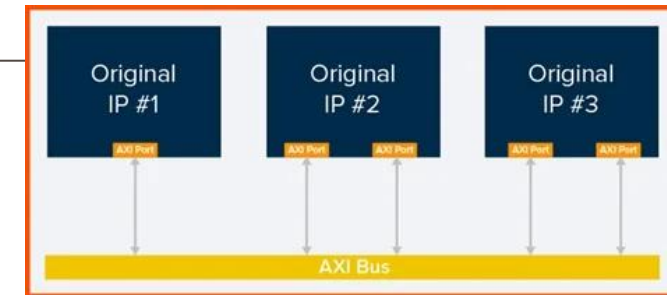
Derivative Designs

- Practically, the architecture for a new system is derived from existing architectures of previous products
- Less than a tenth of a new SoC design will be newly written RTL code
- Time to market is the main driving force

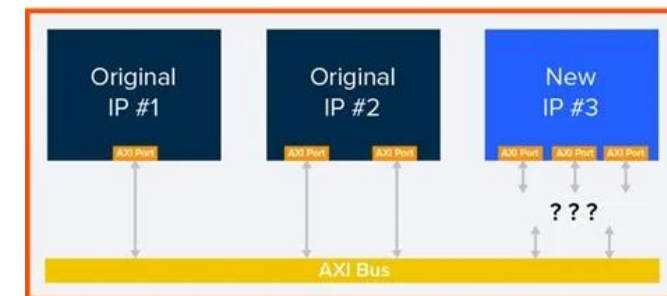
- Several different designs share the same base architecture
 - Same technical specifications (varying in implementations, market needs)
 - Similar requirements

Derivative designs:

- Modify a relatively small portion of a field-proven SoC
 - Can replace/modify one or more functions
 - Keeps the larger proportion of the design intact



(a) Original SoC



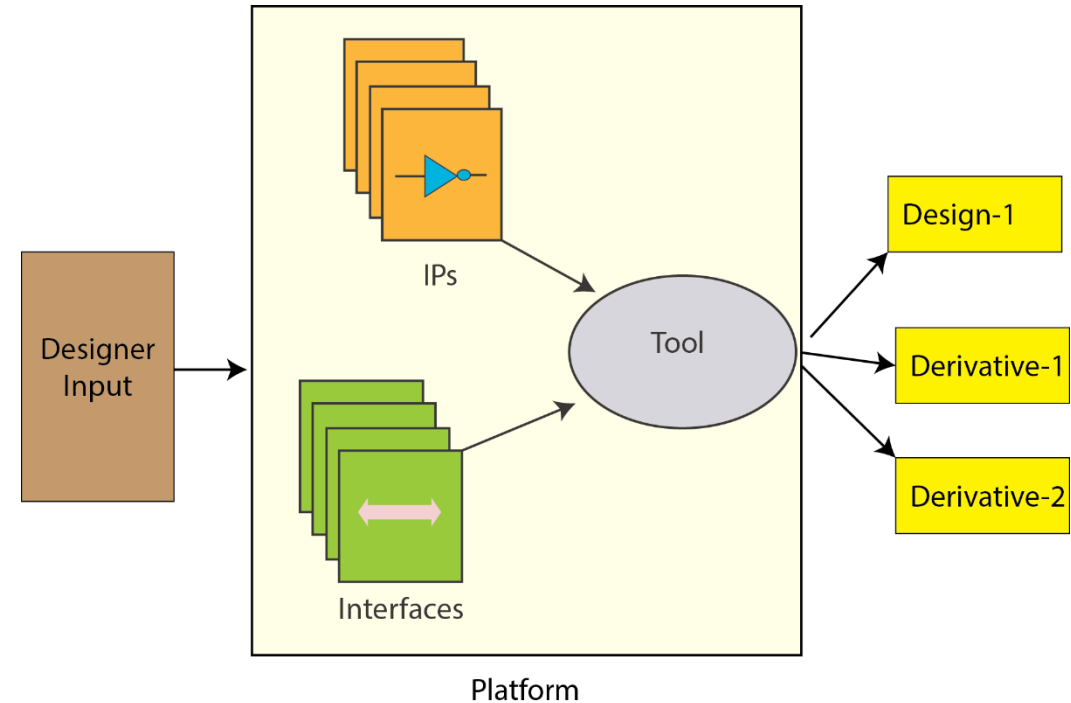
(b) Desired Derivative SoC

<https://www.allaboutcircuits.com/industry-articles/speeding-derivative-soc-designs-with-networks-on-chips/>

Platforms for Derivative Designs: Motivation

Motivation to develop platforms

- Given an application domain, number of good solutions can be limited
 - Find set of good solutions (architectures) by thorough analysis
- Spend more time and money in the development of an architecture that best captures these good solutions
 - Its design cost can be spread over a greater number of end applications
- Set of architectures is called a **platform**



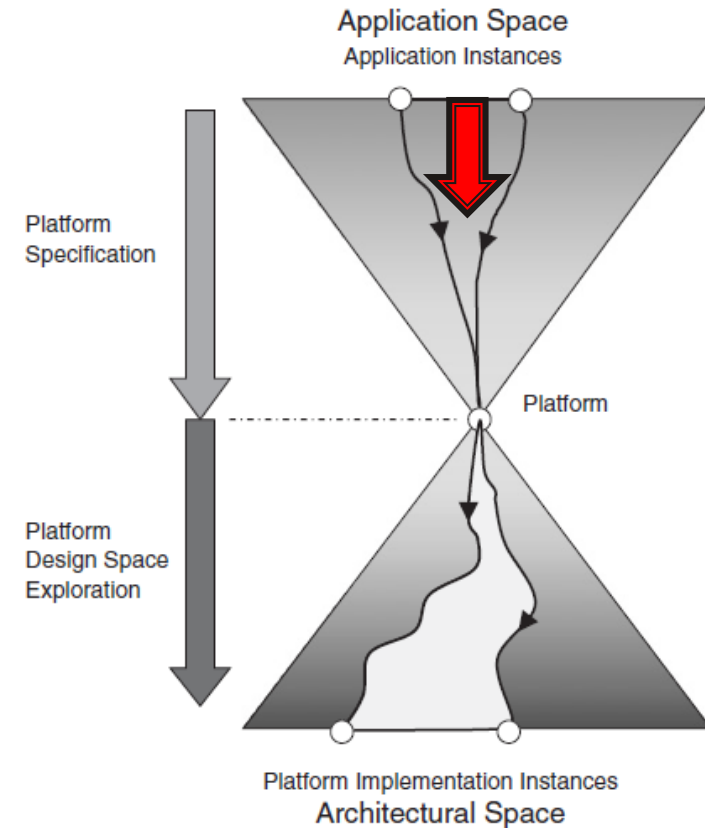
- Platform facilitates reuse of architectures and components
- Provides the application developers with only a limited number of aspects or “knobs” that they can learn to use efficiently.
 - Simplicity is the key requirement for a platform

Platforms: How to Build (1)?

- Platform seen as articulation point between an **application space** and an **architectural space**

Application view (upper view)

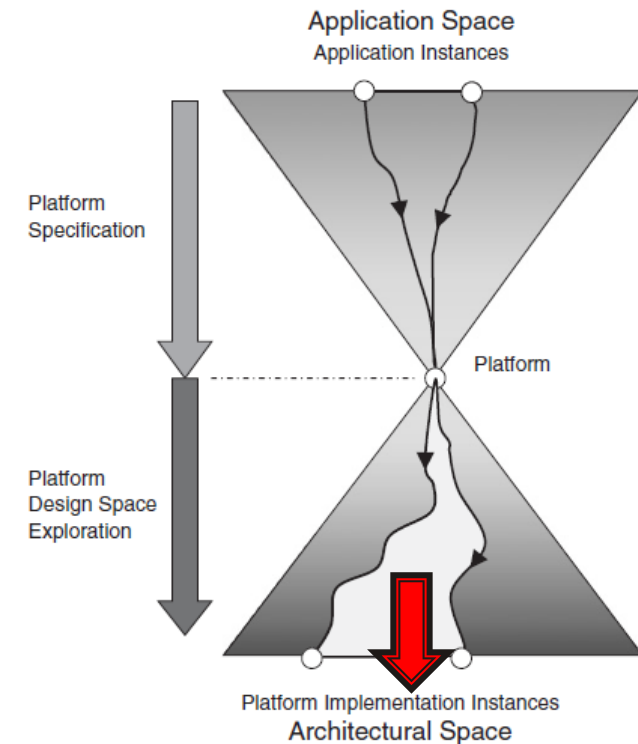
- Details of the implementation abstracted from application developer
 - Eases the development of an application
- Several instances of applications can use the same platform
- Views available:
 - APIs (details of implementation hidden)
 - Instruction set (details of the actual CPU hidden)
 - Device drivers (details of actual peripheral hidden)
 - Operating System (how is concurrency handled is hidden)



Platforms: How to Build (2)?

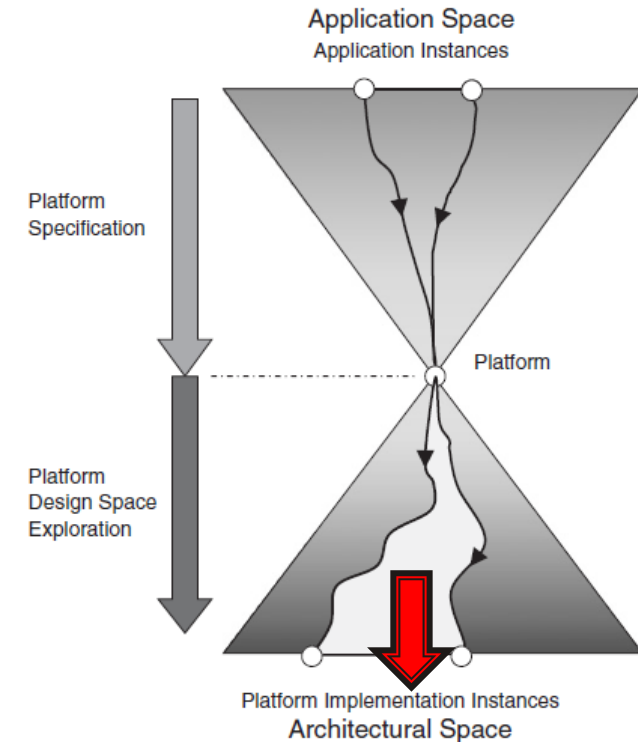
Architectural view or hardware view (lower view)

- Platform is a set of components that can be plugged together to form an SoC
- By implementing a good platform, a designer satisfies the needs of a variety of applications
 - Allows easy design of derivative design
- Platform hides details of the applications that can be mapped on to the hardware
 - Application knows amount of required computational power and communication capability
- Architecture can match the computational power required by an application by:
 - Selecting the appropriate number of processors with the appropriate speed
 - Tune communication bandwidth (insert a certain number of buses with a given bus width, speed, and bus access mode, such as burst access)



Platforms: Link between Application and Architecture

- Platform hides details of the applications that can be mapped on to the hardware
 - Application knows amount of required computational power and communication capability
- Architecture can match the computational power required by an application by:
 - Selecting the appropriate number of processors with the appropriate speed
 - Tune communication bandwidth (insert a certain number of buses with a given bus width, speed, and bus access mode, such as burst access)



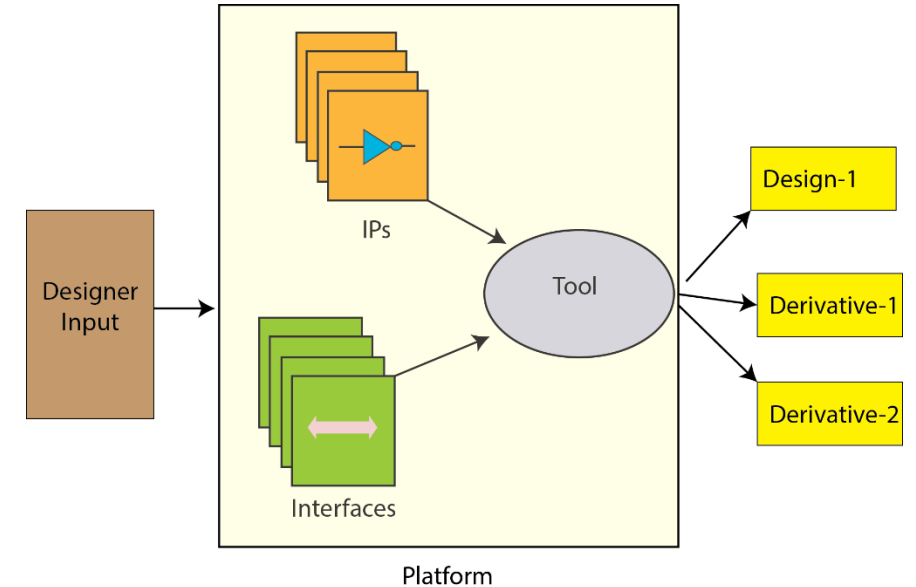
Platform-based Design: Tasks and Infrastructures

Packaging IPs and Interfaces

- Packaging the available architectural components as IPs with a well-defined interfaces
- Packaging a set of interfaces with well-defined communication structures such as buses
- **Desirable:** automatic tools for packaging

IP Assembly

- Choose a set of IPs and interfaces to create an SoC
- Configuring parameters of the IPs
- **Desirable:** automatic tools to create designs and derivatives using designer inputs



Simulation:

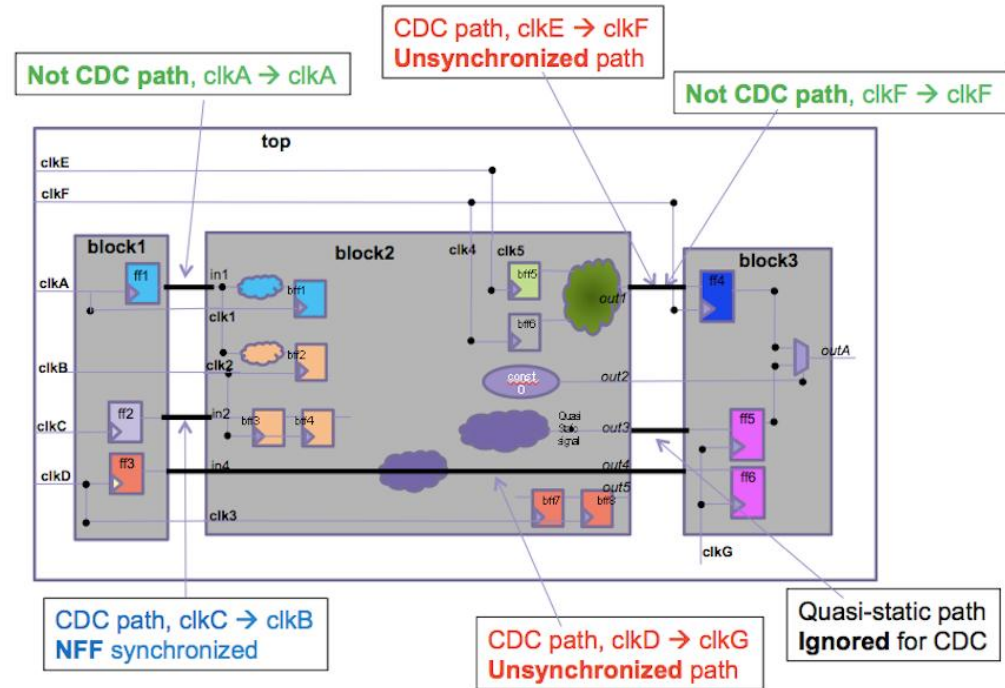
- Quick running of application after customization
 - Instruction set definition at different abstraction levels in SystemC and HDL.
- **Desirable:** fast and flexible

Platform-based Design: Configuring Parameters

- Configuring parameters during IP assembly is a critical task
 - Must be consistent among IPs

Examples of parameter configuration

- Synchronization:** IPs with different time domains (requires the determination of a set of clocks consistent with the IP)
- Memories:** size, address and data width constrained to the application-related selection of peripherals and interface protocols
- Processors and other IPs must be configured for parameters such as the size of register banks, the usage policy and size of a cache.



- Communication mechanisms between the components must be selected consistently
 - On-chip buses, memory-based communication, direct connections, hardware FIFOs, etc.

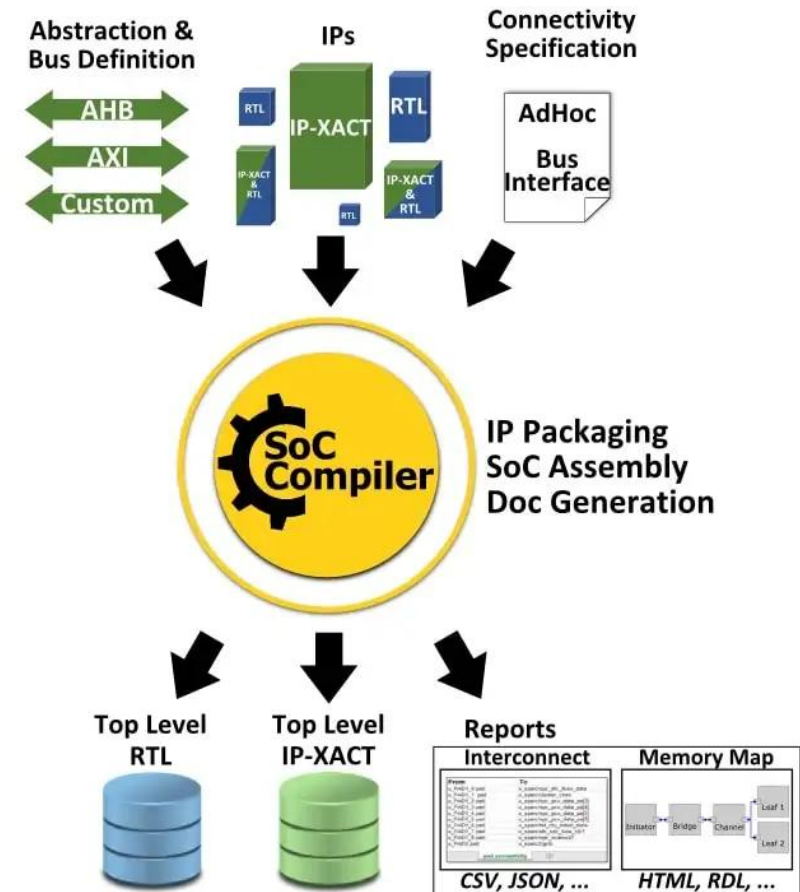
Challenges in IP Assembly and Derivative Designs

Challenges in assembling

- Managing data: high volume and different levels of abstraction
 - Example: interfaces between diverse IP
- Ensuring consistencies among data
 - Example: interfaces between diverse IP
- Producing multiple consistent outputs: RTL, scripts, reports, documentation

Effects

- Challenges in verification:
 - Example: building consistent testbenches for verification
 - Design errors that propagate into prototypes
- Longer time to market



<https://semiwiki.com/eda/314602-using-ip-xact-rtl-and-upf-for-efficient-soc-design/>

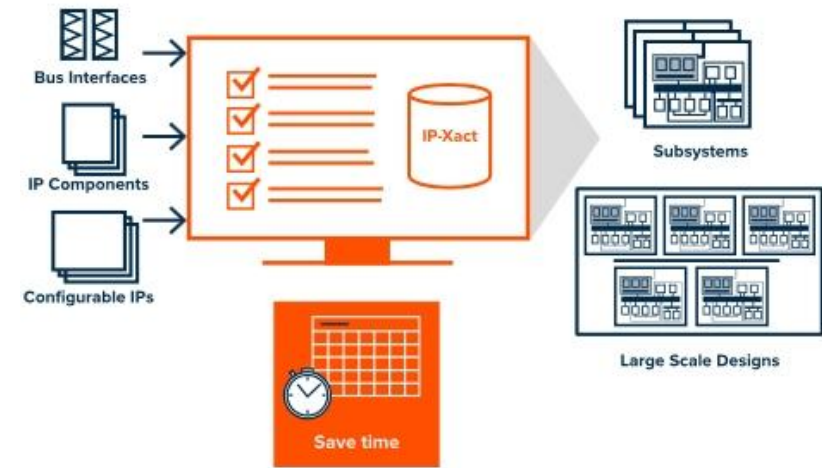
Improving IP Assembly

Streamline IP Assembly

- Providing a common framework for describing IP components
 - Maintain uniformity in the diverse data set
- Standardize data storage and sharing
 - Enhance interoperability between various tools
- Use Design Automation Tools as much as possible
 - Reduce human error
 - Enable automatic verification as much as possible

Solution

- IP-XACT standard (also known as IEEE Std. 1685-2022) is common framework for IP Assembly
- Several tools support this format and enable streamlining IP Assembly and creating derivative designs

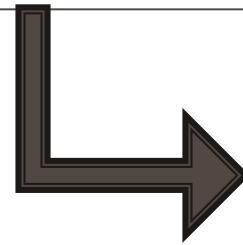


<https://www.design-reuse.com/articles/55064/streamlining-soc-integration-with-the-power-of-automation.html>

Benefits

- Easy to create, package, and reuse IP components using tools
- Minimize errors of integration (improve quality) and improve productivity
- Reduced time-to-market.

SoC Design Methodology



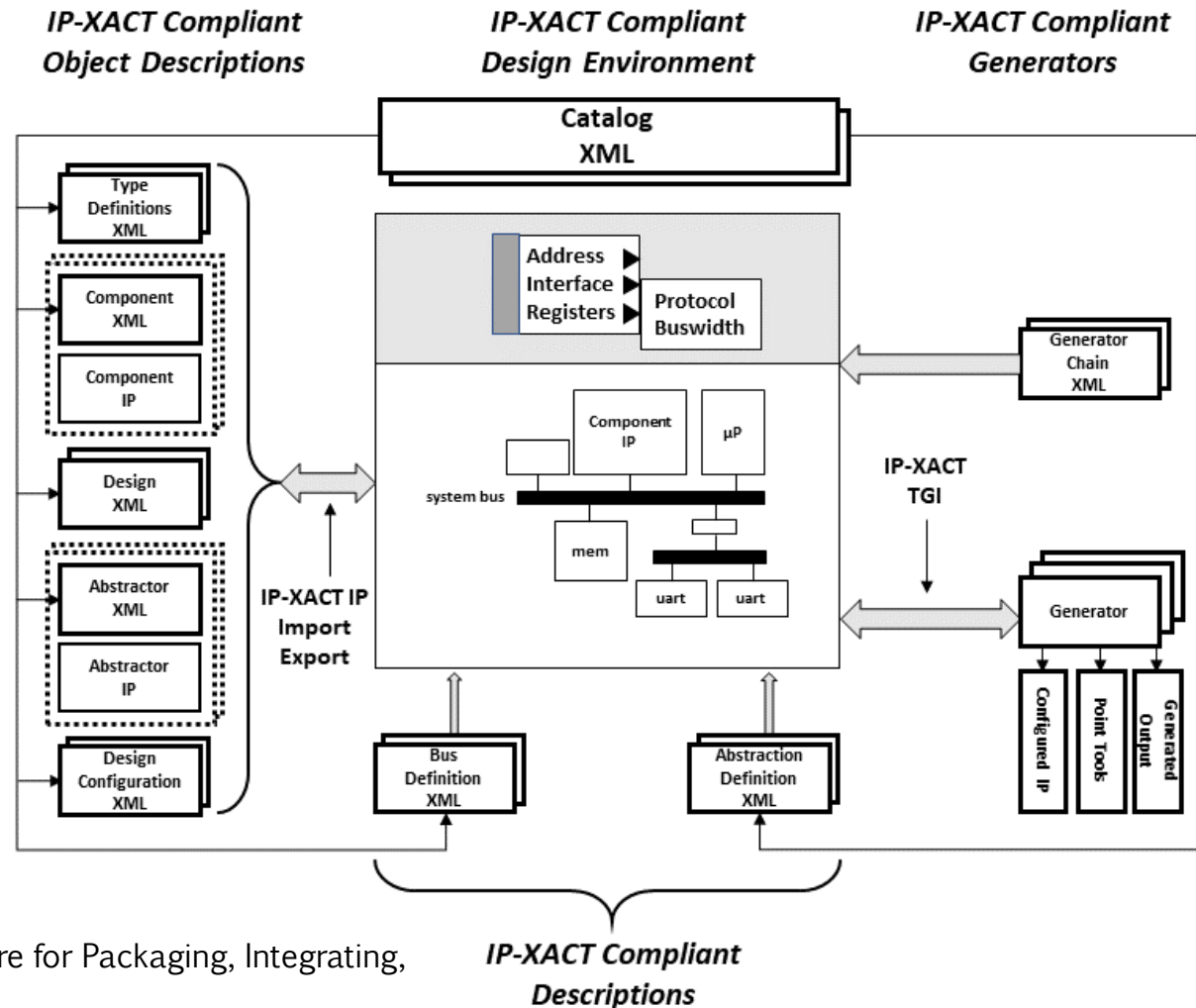
IP-XACT

IP-XACT: Introduction

- IP-XACT stands for Intellectual Property exchange-Advanced Configuration and Integration of IP components

History:

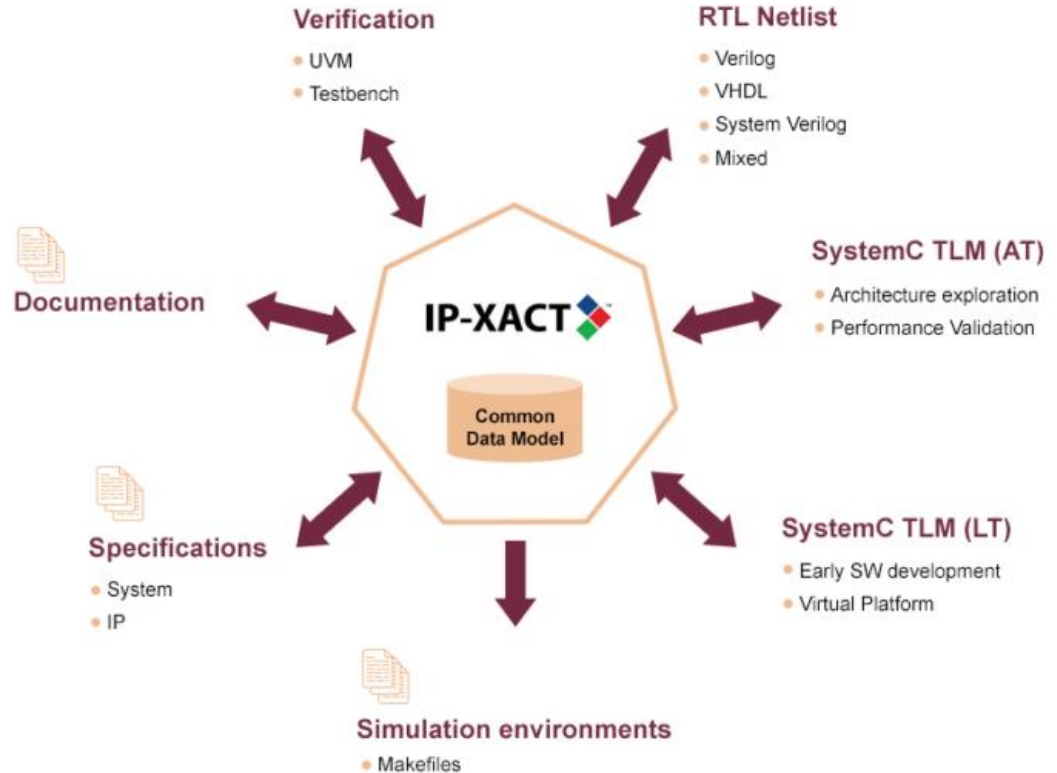
- IP-XACT standard was originally created by the SPIRIT Consortium (which later merged with Accellera)
- The newest revision was released in 2022:
 - IEEE Std. 1685-2022



1685-2022 - IEEE Standard for IP-XACT, Standard Structure for Packaging, Integrating, and Reusing IP within Tool Flows

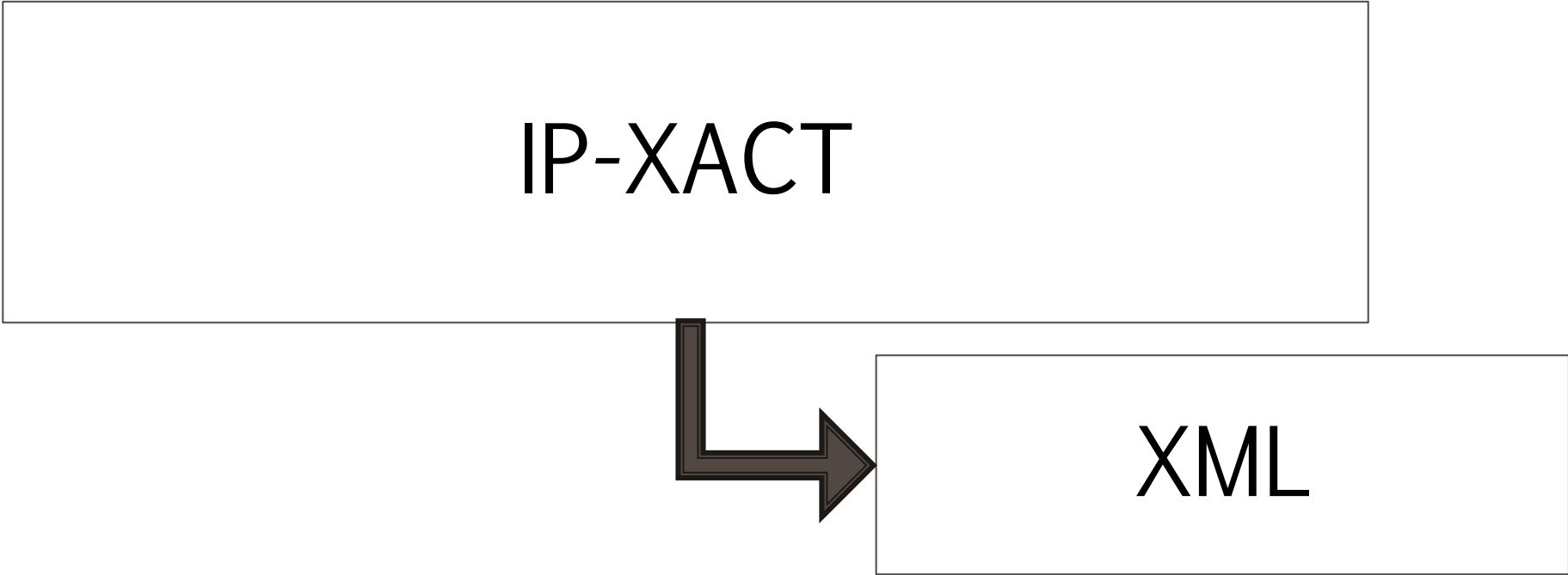
IP-XACT: Salient Features

- **Uniformity:** describes the meta-data of IP designs, flows and the interconnections of interfaces using common data model
 - **Interoperability:** based on XML schema, making it tool-independent
 - **Extendibility:** IP-XACT extension mechanism supports user-defined vendor features to implement specific tool or flow features (without losing compatibility)
-
- IP-XACT streamlines the process of IP packaging, specification (at various levels), integration, verification, and documentation.
 - Improved productivity and reduced time-to-market



cadence® Mentor® A Siemens Business synopsys®

<https://semiwiki.com/eda/magillem/285915-the-life-of-a-soc-designer-is-an-unenviable-one-not-only-does-he-have-to-work-in-a-landscape-where-competition-is-intense-but-he-also-has-to-collaborate-effectively-with-globally-dispersed-teams-to-e/>



```
graph TD; A[IP-XACT] --> B[XML]
```

IP-XACT

XML

XML: Basics

- XML stands for Extensible Markup Language (XML)

- **Purpose:** define and store data in a shareable manner

- **Use cases:** share information between computer systems such as websites, databases, and third-party applications

- XML file is a text-based document that can be save with the .xml extension.
- XML provides rules to define any data.
- XML cannot perform computing operations by itself (unlike programming languages)
 - Software can be implemented for structured data management.

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
<title> Introduction to VLSI Design Flow </title>
<author> Sneh Saurabh </author>
</book>
```

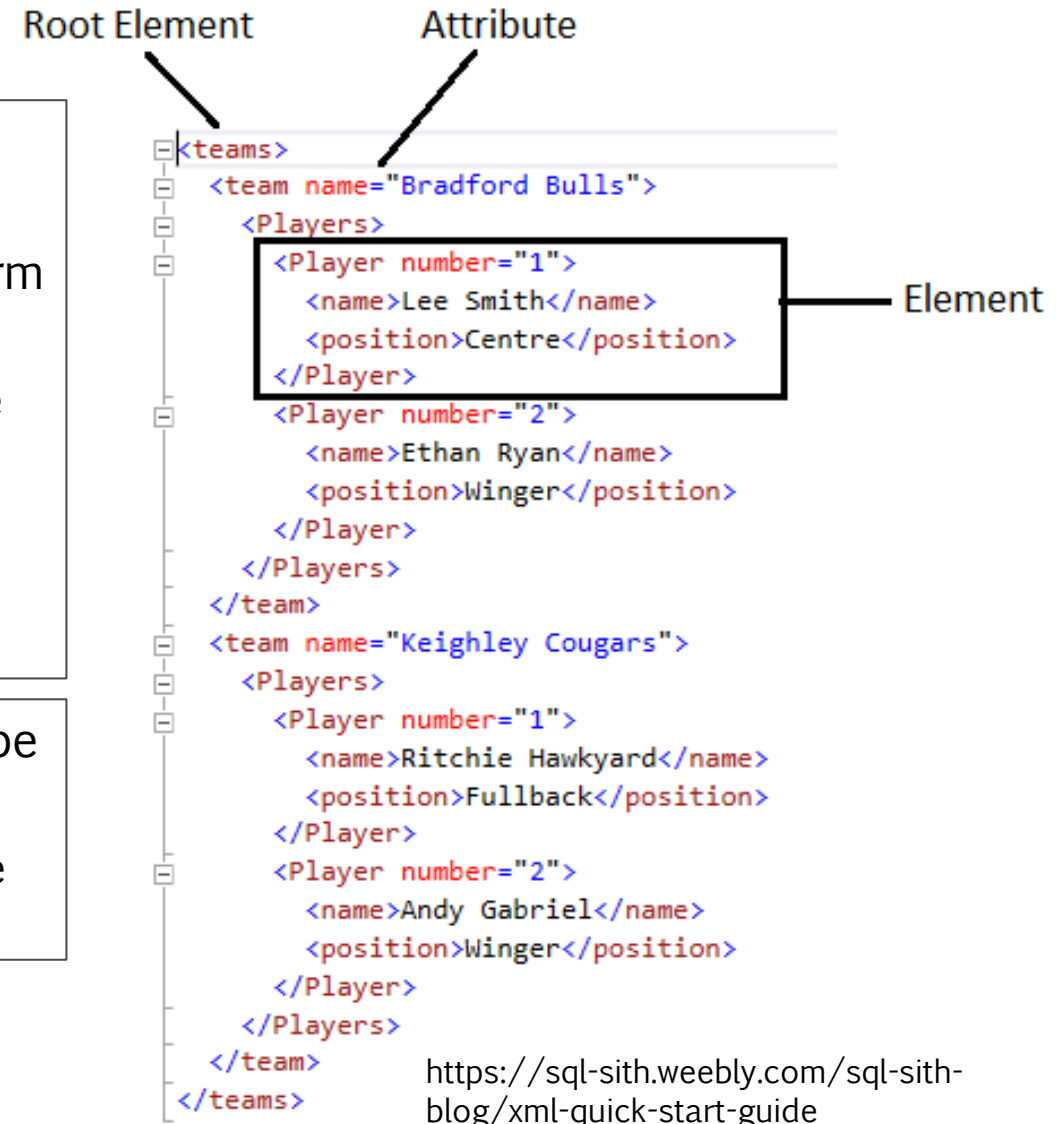
```
-<book>
  <title> Introduction to VLSI Design Flow </title>
  <author> Sneh Saurabh </author>
</book>
```

- **XML declaration:** contains information of XML (version it follows etc.).
- **Tags:** markup symbols to encode data (e.g., <book>, <title> <author>)
- Tags allow integrating information flows across different systems.

XML: Components

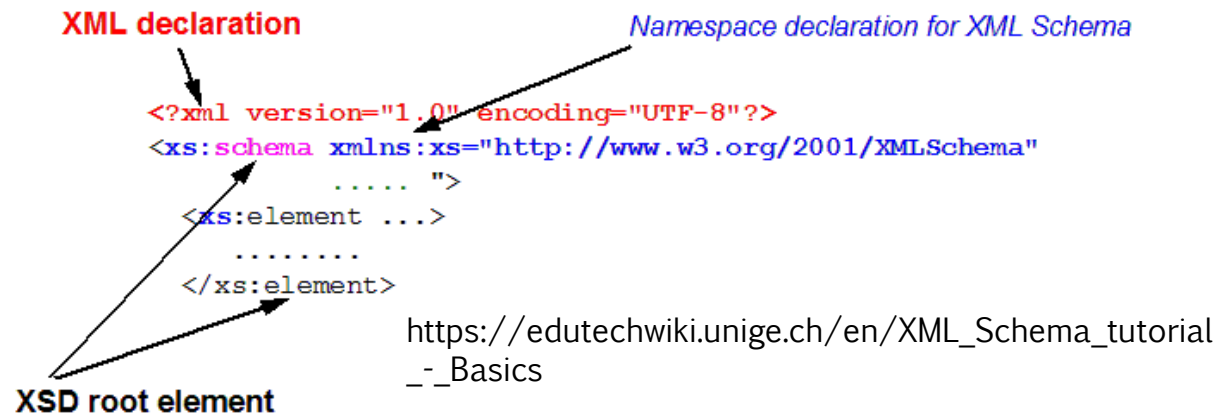
XML documents are composed of elements:

- Contained within a start tag (of the form `<element_name>`) and an end tag (of the form `</element_name>`).
 - Elements can contain other elements or free text.
 - Every XML document has one and only one **root element**, which contains all the other elements.
-
- **Attributes:** are name value pairs that describe an element
 - Example for `<Player>` element: number is the attribute name and “2” is the attribute value



XML: Schema and Parser

- **XML schema:** a document that describes some rules or constraints on the structure of an XML file (pluggable grammar)



- XML schemas enforce consistency in how different software applications create and use XML files.

Constraints can be:

- Grammatical rules to determine the order of elements (e.g.: A contains B)
- Yes or No conditions that the content must satisfy
- Data types for the content in XML files (e.g.: string, integer etc.)
- Constraints for data integrity (e.g.: ranges of values)

- **XML parser:** software that can process or read XML documents to extract the data within them
- Check the syntax or rules of the XML file
- Can validate it against a particular XML schema.

References

- Martin, Grant, Brian Bailey, and Andrew Piziali, “ESL design and verification: a prescription for electronic system level methodology”, Elsevier, 2010.
- Keutzer, Kurt, et al. "System-level design: Orthogonalization of concerns and platform-based design." *IEEE transactions on computer-aided design of integrated circuits and systems* 19.12 (2000): 1523-1543.
- Sangiovanni-Vincentelli, Alberto, and Grant Martin. "Platform-based design and software design methodology for embedded systems." *IEEE Design & Test of computers* 18.6 (2001): 23-33.
- Sangiovanni-Vincentelli, Alberto, et al. "Benefits and challenges for platform-based design." *Proceedings of the 41st Annual Design Automation Conference*. 2004.
- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.
- <https://www.edn.com/system-on-chip-soc-design-is-all-about-ip-management/>
- <https://accellera.org/downloads/standards/ip-xact>